

Correction-conditioned media walls for personal taste alignment

Alan N. Pham^{1,*}

¹AO Labs

*Correspondence: aolabs.io

Personal recommendation systems usually learn taste from ratings, clicks, watch time, purchases, or dense behavioral logs. Yum explores a different unit of personalization: the user’s correction. A correction such as “do not show this model, color, pose, ingredient, person, source family, or same-screen cluster again” is treated as a reusable test that updates a deployed media wall through hard rejects, compact preference memory, source de-duplication, visual-group spacing, car-family spacing, and optional model-assisted candidate ranking. The novelty is not another image feed, but a correction-conditioned control layer for personal AI alignment: free-form dissatisfaction is converted into inspectable feed constraints without requiring a large surveillance history. We describe a live static-first website, yum.aolabs.io, that mixes food, a preference-shaped compact-car lane, and Wonyoung-only K-pop imagery while preserving cross-device preference state and withholding sparse lanes rather than cloning rejected or repeated material. Screenshots of the deployed surface are included as artifact evidence. The system is presented as a design study for preference-aware personal media, not as a population-scale recommendation benchmark.

1 Introduction

Recommender systems have long framed personalization as a prediction problem: infer what an individual will rate, click, purchase, or consume next (1–4). That framing works when the objective is stable and the signal is abundant. It is weaker when the target is a private aesthetic standard that changes through direct correction: “not this color,” “not this kind of image,” “not this person,” “not this texture,” “not this crop,” or “never cluster several similar items again.”

Yum is built around that second setting. It is a personal media wall whose useful behavior is defined less by aggregate engagement than by whether it stops repeating the user’s rejected mistakes. The core design question is therefore not “how can the system maximize interaction?” but “how can a single user’s corrections become durable operational constraints on a live visual artifact?”

This question matters beyond one website. Human-AI systems increasingly depend on natural-language feedback, preference learning, and alignment from human judgments (5–7). However, most deployed personal interfaces still leave users with brittle local settings, opaque ranking models, or short-lived dislike buttons. Yum treats correction as a first-class artifact. A correction is translated into source keys, semantic rejects, group-spacing penalties, category quotas, and preference samples that are small enough to inspect and persistent enough to matter across sessions.

2 Novelty

2.1 From implicit feedback to explicit correction

Implicit-feedback recommenders infer taste from behavior that was not necessarily intended as feedback (3, 4). A scroll, linger, or click can be ambiguous. Yum instead elevates direct correction. The user may reject an exact item, but the implementation must infer the reusable failure mode: the source may be low-quality, the category may be overrepresented, the subject may be repeated, the image may violate an aesthetic rule, or a cluster may be visually monotonous.

The proposed novelty is *correction-conditioned curation*: a deployed feed architecture in which free-form human correction is compiled into a transparent constraint stack. The stack has four properties.

1. It is **persistent**: a correction can affect a later page load and a different device through remote preference state.
2. It is **inspectable**: hidden keys, hidden samples, kept samples, blocked terms, and group identifiers are ordinary data rather than an opaque behavioral archive.
3. It is **compositional**: hard rejects, category balance, source rotation, model ranking, and layout placement each address a different failure mode.
4. It is **artifact-facing**: the final test is not an offline score but whether the public wall stops showing the rejected pattern.

2.2 Why this is different from a dislike button

A dislike button removes an item. A correction-conditioned system must generalize without over-generalizing. If the user rejects a blurred K-pop image, the system should prefer higher-resolution images without eliminating the subject. If the user identifies a specific configured vehicle as especially desirable, the system should keep that build as a high-confidence anchor without collapsing the entire car lane into one vehicle. If several images from one source or one car family appear together, the system should treat the cluster as a layout and queueing failure rather than a single bad tile.

This makes the central object of study a feedback-to-artifact loop. The user does not need to become a taxonomy editor. The system must convert correction into durable tests and then verify the deployed surface against those tests.

3 System design

Yum is a static-first site with optional dynamic services. The public wall is served at `yum.aolabs.io`. The frontend contains known-good seed media, source metadata, category labels, captions, source URLs, shape hints, focus hints, and optional person, visual-group, or car-family identifiers. In the current deployed build, the car lane keeps Audi Code A0J42547, an A3 TFSI quattro Premium Plus S tronic in Manhattan Gray metallic with Black optic package and Parchment Beige–Steel Gray interior, as an approved Audi A3 family, adds official 2025/2026 MINI Cooper 2 Door/Hardtop imagery, and removes BMW-branded vehicles, MINI Countryman/crossover substitutions, and non-A3 Audi vehicles from the allowed mix. The Audi gate is literal: A4, A5, S3, RS3, Q-series, generic Audi sedans, large Audi cars, and any non-A3 Audi are rejected even when the images are official. Vehicle candidates pass both a composition gate and a same-screen family gate. The composition gate rejects side-only profiles, cabin and detail crops, wheel or trim fragments, missing local derivatives, known source identifiers whose actual image contradicts its exterior caption, and margin-prone frames that would render as a small car inside a dark tile. The family gate treats A0J42547/Audi A3, MINI Cooper 2 Door/Hardtop, and Mercedes-Benz compact-sedan batches as source families that cannot repeat inside the same visible screen. K-pop candidates now pass an adult-era body-and-styling gate centered on Wonyoung only. Karina, Yujin, Kazuha, Hanni, Haerin, Ningning, Aespa, and Le Sserafim sources are rejected before display and before backend candidate selection. The K-pop fallback is visual-group unique, so one source shoot or outfit cannot be cloned into several visible tiles while the online refill is still loading. Food candidates pass a composition gate that rejects vegetable spreads, tomato, pumpkin, vegetable, and plain soup shots, plain garnish or ingredient bowls, empty-plate shots, restaurant exterior or sign photos, lunch boxes, packaged meals, visible people, family meals, children, dining-room scenes, home or office food snapshots, people-at-table scenes, dim bad-framed documentation, and weak dining-documentation frames; food now cycles from the reviewed static pool rather than live online food refills. A small Node service provides preference persistence, model-assisted K-pop and car candidate ranking, and Wonyoung candidate retrieval. The static wall remains useful if the dynamic service is slow or unavailable.

3.1 Taste state

91

Preference state is intentionally compact. It contains hidden source keys, hidden samples, kept samples, a version number, and timestamps. Hidden keys remove exact items. Hidden samples provide stronger negative examples. Kept samples provide weak positive context from nearby non-hidden material. This is a privacy-leaning alternative to large behavioral histories: the state is small, inspectable, and biased toward explicit correction rather than passive surveillance.

96

3.2 Candidate control

97

Candidate generation and candidate control are separated. Candidate generation finds possible images from static pools and remote sources. Candidate control decides what is allowed to reach the wall. Control includes blocked terms, source de-duplication, category balancing, recent-person spacing, visual-group spacing, car-family spacing, quality thresholds, composition gates, natural-aspect tile sizing, and optional language-model ranking. For the present vehicle preference, the car lane allows official MINI Cooper 2 Door/Hardtop imagery, Audi A3 imagery, and modern compact Mercedes sedans while rejecting BMW-branded vehicles, non-A3 Audi vehicles, MINI Countryman/crossover substitutions, SUVs, fire trucks, old cars, disliked color/model clusters, adjacent non-sedan body styles, boring side-profile product renders, crop-heavy fragments where the full car is not visible, and source frames whose dimensions would create visible letterboxing. Exact URL uniqueness is not sufficient: all A0J42547/Audi A3 angles are one Audi A3 family, repeated Mercedes-Benz batches are one Mercedes-Benz family, and MINI Cooper 2 Door/Hardtop images are one MINI family until the user approves finer grouping. The same curator gate is applied to local seed items, queued refills, remote candidates, stale queue entries, and loose-fill fallbacks so old pools cannot bypass newer taste constraints. If a category stalls because no fresh approved family or source shoot is available, the wall withholds that lane and lets approved food fill the screen rather than cloning a car family, cloning one K-pop outfit, or substituting an unapproved idol.

114

This separation is important because visually bad outcomes often arise after a candidate has passed a relevance check. Four similar cars in one view, three portraits of the same person, or repeated low-resolution source patterns are layout-level failures. They must be handled by queue and placement logic, not only by candidate retrieval.

118

4 Deployed artifact

The deployed wall is shown in Fig. 1 and Fig. 2. The screenshots show the actual interaction surface: a paper link, the wall itself, mixed subject categories, and a responsive masonry layout.

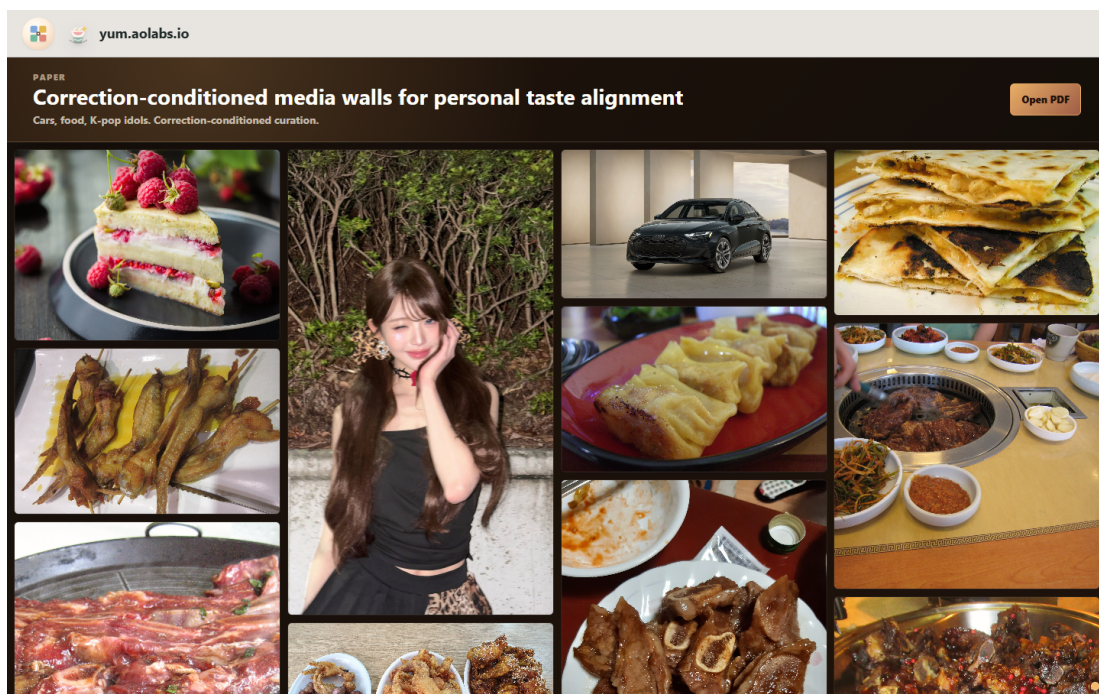


Figure 1. Desktop view of the Yum wall after the same-screen family gate. Car tiles use the loaded image’s natural aspect ratio and cover the tile without artificial dark margins, while the queue and placement logic prevent one car family, source batch, or Wonyoung source shoot from clustering in the visible viewport.



Figure 2. Narrow viewport view of the deployed wall after the same source-family gate is applied at a different column count.

5 Relationship to prior work

Yum builds on recommender-system work but shifts the evaluation target. Classic surveys and hybrid recommender methods emphasize prediction, filtering, and combining sources of evidence (2, 8). Implicit-feedback methods formalize preference from observed behavior (3, 4). Visual recommendation work extends this to appearance-sensitive domains (9). Human-centered recommender evaluation argues that perceived quality, variety, effort, and privacy affect system experience, not just predictive accuracy (10).

The correction-conditioned wall fits between these traditions and human-AI interaction. Human preference learning and instruction-following models demonstrate that human judgments can train or steer model behavior (5, 6). Human-AI guidelines emphasize intelligibility, controllability, and graceful handling of user feedback (7). Vision-language models show that natural language can name visual concepts and support flexible image understanding (11). Yum applies these ideas at the product-surface scale: the user provides corrections in ordinary language, and the live artifact is updated through explicit feed constraints.

6 Implications

The strongest implication is that small personal systems can be aligned through correction memory without becoming black-box surveillance systems. A personal media wall does not need to store every scroll event to improve. It can store what the user explicitly rejected, what remained nearby, and how those examples map to transparent constraints. This produces a more repairable system: when the wall is wrong, the correction can become a test.

The second implication is that personalization should be evaluated at the artifact boundary. A recommender may rank candidates correctly in isolation while still producing a bad page because similar items cluster, a category disappears, or a visual pattern repeats. Yum therefore treats the rendered wall as the unit of verification.

7 Limitations

This manuscript reports a deployed design study, not a controlled user study. The current evidence is artifact-level: source code, deployed behavior, screenshots, and live verification. No population-scale

preference model, benchmark dataset, or statistical generalization claim is made. The system also inherits the limitations of third-party image sources, including uneven metadata quality, unavailable images, source licensing constraints, and inconsistent visual resolution.

The next useful step is not to add more opaque intelligence. It is to measure correction-to-fix latency: after a user gives a rejection pattern, how quickly does the live wall stop violating it, and how often does the generalization accidentally suppress material the user would have liked?

8 Conclusion

Yum argues for correction-conditioned curation as a practical unit of personal AI alignment. A user should not need to repeatedly reject the same failure mode. The system should convert correction into persistent, inspectable constraints and verify the live artifact against those constraints. For personal visual media, this may be a more useful target than engagement maximization: the wall becomes better when it remembers what the user meant, not merely what the user clicked.

References

1. P. Resnick, H. R. Varian, Recommender systems. *Communications of the ACM* **40**, 56–58 (1997).
2. G. Adomavicius, A. Tuzhilin, Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering* **17**, 734–749 (2005).
3. Y. Hu, Y. Koren, C. Volinsky, presented at the 2008 Eighth IEEE International Conference on Data Mining, pp. 263–272.
4. S. Rendle, C. Freudenthaler, Z. Gantner, L. Schmidt-Thieme, presented at the Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence, pp. 452–461.
5. P. F. Christiano *et al.*, presented at the Advances in Neural Information Processing Systems, vol. 30.
6. L. Ouyang *et al.*, presented at the Advances in Neural Information Processing Systems, vol. 35, pp. 27730–27744.
7. S. Amershi *et al.*, presented at the Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, pp. 1–13.

8. R. Burke, Hybrid recommender systems: survey and experiments. *User Modeling and User-Adapted Interaction* **12**, 331–370 (2002). 176
177
9. R. He, J. McAuley, presented at the Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, pp. 144–150. 178
179
10. B. P. Knijnenburg, M. C. Willemsen, Z. Gantner, H. Soncu, C. Newell, Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction* **22**, 441–504 (2012). 180
181
182
11. A. Radford *et al.*, presented at the Proceedings of the 38th International Conference on Machine Learning, pp. 8748–8763. 183
184

Methods 185

Artifact capture 186

Desktop and narrow-viewport screenshots were captured from the current verification build with headless Chrome before publication. The current desktop figure was captured on June 27, 2026 and stored as `assets/yum-wall-desktop-20260627-current.png`; the narrow-viewport figure was captured on June 27, 2026 and stored as `assets/yum-wall-mobile-20260627-current.png`. 187
188
189
190

Static surface and media metadata 191

The public surface is generated from a static bundle containing the HTML shell, CSS, JavaScript wall renderer, icon assets, preview images, paper files, and figure assets. Media candidates are represented as structured entries with image source, source URL, caption, category, tile shape, focus hint, and optional person, visual-group, or car-family metadata. 192
193
194
195

Preference storage 196

Preference state is stored as a compact object containing hidden keys, hidden samples, kept samples, a version number, and update metadata. The browser keeps a local mirror for immediate interaction. The backend stores the same preference shape so state can be restored on another device when the remote service is reachable. 197
198
199
200

Candidate ranking and layout

Candidate ranking is static-first. Known-good seed media is available before any remote service responds. Optional curation requests can pass candidate metadata and preference samples to the configured model. Blocked terms, source keys, category balance, visual groups, person groups, car groups, car families, quality thresholds, and recent placement history are then applied before display. The renderer preloads and decodes candidate images off the visible wall through a bounded worker queue, validates the loaded dimensions, and appends only usable loaded images to the masonry columns. Failed or slow remote images therefore cannot occupy hidden card shells, create empty lanes, or make the wall believe it is full. The first visible batch uses smaller remote image requests where the source supports resizing, and the scroll loader uses a shorter load-ahead window so the browser does not flood itself with hundreds of off-screen image requests. Incremental placement reads the current column pixel heights and fills the shortest visible column first, while loose-fill can only use category items that still pass the same family gates. Scroll checks are throttled through an animation-frame gate, and the tile styling avoids always-on image filters, oversized shadows, and compositor-forcing transforms. The refill trigger counts live loaded tiles and a bounded active pending buffer, not only intended records, so rejected images cannot make the wall believe it is full and pending images cannot flood the browser before paint. The car refill pool uses the curated Mercedes plus MINI 2 Door and Audi A3 set, including the A3 build derivatives, rather than unrestricted vehicle search, BMW-brand fallbacks, or non-A3 Audi sources; the display gate then spaces those by car family so repeated A3 angles and repeated Mercedes-Benz batches cannot occupy one screen. The K-pop refill prompt now requires Wonyoung-only crop-top, exposed-waist, midriff, belly, navel, or body-visible adult-era pose evidence; bare-shoulder-only frames, legacy Hanni and Haerin portrait sources, Karina, Yujin, Kazuha, Aespa, Le Sserafim, face-only crops, forehead-heavy closeups, repeated same-outfit source runs, outerwear-heavy frames, and minor-era styling are rejected. The K-pop queue records visual groups as session-used groups, so a second image from the same Wonyoung source shoot cannot enter the rendered wall merely because it has a different URL. Food no longer refills from live online searches because broad Commons and search results repeatedly admitted family-table, leftover-plate, and vegetable-heavy documentation frames even when metadata looked acceptable. If the food lane stalls, the wall cycles same-category reviewed food seeds instead. This keeps the wall from terminating into blank space without allowing one unreliable category to overrun the others.

Production synchronization

231

The deployed system has two independently updated surfaces: the GitHub Pages static bundle at yum.aolabs.io and the Railway Node service that serves preference and candidate APIs. A correction is not considered shipped until both surfaces are current. The May 19, 2026 production check found a split state: the static bundle already served the v41 correction gates, but the Railway /api/kpop-candidates endpoint still returned stale Hanni Fashion Week, Hanni 2022, Haerin Fashion Week, and Wonyoung 220513 candidate sources. The Node service was redeployed to Railway as deployment b705079c-e3a7-4b57-8a45-1ba2a4a1d3f4. Direct API queries for Hanni, Haerin, and Wonyoung then returned zero hits for those rejected source families, while the live JavaScript bundle still contained the outerwear gate and refill heartbeat and omitted the old Fashion Week and Ningning static seeds. The June 27, 2026 correction adds the same production rule for source-family repetition and idol scope: frontend, backend candidate retrieval, static bundle, paper, and rendered verification must all agree before the correction is treated as shipped.

232
233
234
235
236
237
238
239
240
241
242
243

Verification

244

The manuscript PDF was generated from this LaTeX source using pdf_lat_ex and biber. The deployed site was verified by checking that the live paper URL returned a PDF, that the live page served the current JavaScript bundle, that the bundle contained the current correction-conditioned curation rules, that desktop and narrow live viewports contained no repeated car family, rejected idol, non-A3 Audi, BMW, or visible unloaded tile, and that the Railway candidate API no longer returned the source families removed by the correction gates.

245
246
247
248
249
250

Data availability

251

No new research dataset was generated. The media metadata required to inspect the public wall is included in the deployed JavaScript bundle and in the Yum application source. The current car lane combines local image derivatives extracted from a user-supplied Audi configuration artifact for Audi Code A0J42547 with restored third-party modern-sedan sources. Third-party media remains subject to the licensing and availability terms of the original sources linked from the wall.

252
253
254
255
256

Code availability

257

The application code and this LaTeX source are available in the Yum application repository at github.com/nalalalan/yum-app. The deployed LaTeX source is served at yum.aolabs.io/yum-preference-shaped-media-wall.tex.

Acknowledgements

No external acknowledgements are declared.

Funding statement

No external funding is declared.

Author contributions

A.N.P. conceived the system, implemented the wall and backend, designed the curation and preference-memory behavior, prepared the manuscript, and maintains the public deployment.

Competing interests

A.N.P. created and operates AO Labs and Yum.

Additional information

Supplementary information is not included. Correspondence and requests for materials should be directed to Alan N. Pham through aolabs.io.